

DNA BJJ App

Frontend ↔ Backend Integration Roadmap

A complete map for connecting two separate GitHub repos into one working app

The Big Picture

The frontend and backend live in separate GitHub repos. They do not need to be merged — they stay separate forever. They communicate over HTTP: the React frontend makes API calls to the Express backend. The backend talks to Supabase. That's the whole system.

Repo	What it does
dna-bjj-frontend	The React app the coach uses. Runs in the browser on the tablet.
dna-bjj-api	The Express backend. Handles data, auth, and talks to Supabase.

💡 *These repos never merge. They just need to know each other's URLs.*

How they connect:

```
[ Coach's Tablet — Browser ]
    ↓ HTTPS API calls
[ Railway — Express API (dna-bjj-api) ]
    ↓ Supabase SDK
[ Supabase — Database + Auth ]
```

Deployment Plan

Each repo deploys independently to its own hosting service:

Repo	Hosting Service	Result
dna-bjj-api (backend)	Railway	A live API URL, e.g. https://dna-bjj-api.railway.ap p

dna-bjj-frontend (frontend)	Vercel or Netlify	A live app URL, e.g. https://dna-bjj.vercel.app
--------------------------------	-------------------	---

💡 *Railway is already chosen for the backend. Vercel is the easiest choice for the React frontend — free tier, deploys on every GitHub push, zero config for React apps.*

Step-by-Step Roadmap

Complete these in order. Each step unblocks the next.

1

Deploy the Backend to Railway

dna-bjj-api repo · ~30 minutes

1. Go to railway.app and sign in with GitHub
2. Click New Project → Deploy from GitHub repo → select dna-bjj-api
3. Railway will auto-detect Node.js and run npm start
4. Go to your project → Variables tab → add these environment variables:

```
SUPABASE_URL           = your Supabase project URL
SUPABASE_SERVICE_ROLE_KEY = your service role key
PORT                   = 3000
```

5. Once deployed, Railway gives you a public URL — copy it. It looks like:
<https://dna-bjj-api-production.up.railway.app>
6. Test it by visiting that URL + /health in your browser — you should see { status: 'ok' }

💡 *The backend must be live and responding before you touch the frontend.*

2

Create the Coach Account in Supabase

Supabase dashboard · 5 minutes

7. Go to your Supabase project → Authentication → Users
8. Click Add User → enter the coach's email and a strong password
9. That's it — this is the only account needed since the app is coach-only

💡 *The coach will use this email/password to log into the app on the tablet.*

3

Add Auth + API Config to the Frontend*dna-bjj-frontend repo · Claude Code*

This is the main wiring step. In Claude Code, from the dna-bjj-frontend root, run:

Ask Claude Code to:

- Install `@supabase/supabase-js` on the frontend
- Create `src/lib/supabase.js` using the anon key (not the service role key)
- Create `src/lib/api.js` — a fetch wrapper that attaches the JWT token to every request as a Bearer header and points to the Railway URL
- Create `src/pages/Login.jsx` — email + password form, calls Supabase auth, stores the token
- Update `App.jsx` React Router so every route except `/login` redirects to `/login` if no token
- Add a logout button that clears the token

Key	Value
Frontend uses (Supabase)	anon public key — safe to expose in the browser
Backend uses (Supabase)	service role key — never expose this on the frontend
API base URL	Your Railway URL from Step 1

4

Replace localStorage with Real API Calls*dna-bjj-frontend repo · Claude Code*

Once auth is working, replace every `localStorage` call in these files:

File	What changes	API route it calls
<code>Students.jsx</code>	Load and add students	GET <code>/students</code> , POST <code>/students</code>
<code>StudentProfile.jsx</code>	Edit, delete, award stripes	PATCH <code>/students/:id</code> , DELETE <code>/students/:id</code> , POST <code>/stripes</code>
<code>Attendance.jsx</code>	Check-in, view history	POST <code>/attendance</code> , GET <code>/attendance</code>
<code>Waivers.jsx</code>	Display waiver status only	GET <code>/students</code> (waiver_active field)

💡 Do one file at a time and test after each one. *Students.jsx* first — it's the core of the app.

5

Deploy the Frontend to Vercel

dna-bjj-frontend repo · ~15 minutes

10. Go to vercel.com and sign in with GitHub
11. Click Add New Project → Import dna-bjj-frontend
12. Vercel auto-detects React — no build config needed
13. Add one environment variable in Vercel's dashboard:
`VITE_API_URL = https://your-railway-url.up.railway.app`
14. Deploy — Vercel gives you a public URL for the app
15. Open that URL on the iPad and the Samsung Tab to confirm it works on both

💡 Every time you push to main on GitHub, Vercel auto-redeploys. No manual steps after initial setup.

6

End-to-End Test

Both devices · ~20 minutes

Run through this checklist on the Samsung Galaxy Tab (the real coach device):

- Open the Vercel URL in Chrome on the tablet
- Log in with the coach account created in Step 2
- Add a test student — confirm they appear in Supabase
- Check the student in for attendance — confirm the row appears in the attendance table
- Award a stripe — confirm it appears in the stripes table
- Log out and confirm you're redirected to the login screen
- Add the app to the home screen (Chrome → Share → Add to Home Screen)

One Thing to Watch Out For — CORS

When the frontend (on Vercel) makes API calls to the backend (on Railway), the browser will block them unless the backend explicitly allows it. This is called CORS.

Before deploying, add this to the backend's `index.js`:

```
import cors from 'cors';  
app.use(cors({ origin: 'https://your-vercel-url.vercel.app' }));
```

And install it:

```
npm install cors
```

💡 Without this, every API call from the frontend will fail silently in the browser. Add it before Step 5.

Quick Reference Checklist

Step	Task	Status
1	Deploy dna-bjj-api to Railway	☐ Not done
2	Create coach account in Supabase	☐ Not done
3	Add auth + API config to frontend	☐ Not done
4	Replace localStorage with API calls	☐ Not done
4a	Add cors to backend index.js	☐ Not done
5	Deploy dna-bjj-frontend to Vercel	☐ Not done
6	End-to-end test on Samsung Tab	☐ Not done

The two repos stay separate forever — they just talk to each other over HTTP. That's it.